Universidad del Istmo
FING
Víctor Vargas
Ingeniería Electrónica y Telecomunicaciones



Examen Final

Puntos extra: Memoria parametrizada en SystemVerilog

Sofia del Pilar Martínez Chajón

1. Introducción

Este informe documenta la implementación, en SystemVerilog, de la estructura de memoria explicada en el video **“How Transistors Remember Data”** (canal Core Dumped). El video construye una memoria paso a paso: parte de una compuerta lógica realimentada que “recuerda” un bit, llega a una matriz de celdas seleccionadas por decodificadores y termina en una memoria de bytes como la que usa un procesador.

El diseño reproduce esa misma jerarquía a nivel de compuertas y la reúne en un módulo top parametrizado, `memory_top`, cuyo ancho de palabra (`DATA_WIDTH`) y número de direcciones (`2^ADDR_WIDTH`) se eligen al instanciarlo. El funcionamiento se verificó con testbenches autoverificables en Icarus Verilog 12 y se observó con GTKWave.

2. Objetivos

- Implementar en SystemVerilog cada bloque descrito en el video: AND-OR latch, gated latch, registro, decodificador binario, celda de memoria, matriz y memoria de palabras.
- Integrar todos los bloques en un módulo top **parametrizado** (`memory_top`).
- Verificar por simulación la escritura, la lectura, el aislamiento entre direcciones y el bus en alta impedancia, para varios tamaños de memoria.
- Explicar el funcionamiento interno de la memoria a partir de las formas de onda obtenidas.

3. Marco teórico

3.1 Realimentación: cómo un circuito recuerda

Una compuerta lógica combinacional no tiene memoria: su salida depende sólo de sus entradas en ese instante. Para conservar información es necesario **realimentar** la salida hacia una de las entradas. Si la salida de una compuerta OR se conecta a su propia entrada, basta un 1 momentáneo en la otra entrada (SET) para que la salida quede en 1 de forma permanente, aunque SET regrese a 0: el circuito “recuerda” que en algún momento recibió un 1.

3.2 AND-OR latch

El OR realimentado no puede volver a 0. Para borrarlo se coloca una compuerta AND en la salida, cuya segunda entrada es la señal RESET invertida. Con RESET = 1 la AND fuerza la salida a 0 y, como esa salida también se realimenta, el 0 se mantiene. El comportamiento queda descrito por:

$$Q^{+} = (Q + SET) \cdot RESET'$$

Tabla 1. *Tabla de verdad del AND-OR latch.*

SET	RESET	Q^+	Efecto
0	0	Q	Mantiene el valor guardado
1	0	1	Escribe un 1 (set)
0	1	0	Escribe un 0 (reset)
1	1	0	RESET domina (combinación que se evita)

3.3 Gated latch

Manejar SET y RESET por separado es poco práctico. El gated latch agrega lógica de entrada para usar **una sola línea de dato** y una señal de habilitación de escritura (WE):

$$\mathbf{SET = D \cdot WE} \qquad \mathbf{RESET = D' \cdot WE}$$

Si WE = 0, SET y RESET valen 0 y el latch conserva su valor sin importar D. Si WE = 1, la salida copia el valor de D. Además, SET y RESET nunca pueden valer 1 al mismo tiempo. Este bloque es la celda básica de 1 bit.

3.4 Registro

Colocando N gated latches en paralelo con una línea WE común se obtiene un **registro** de N bits; por ejemplo, el valor 92 se guarda como 01011100 en un registro de 8 bits. Así son los registros de propósito general y el registro de instrucción de un CPU. Su limitación es que cada bit necesita su propio cable de entrada y de salida, por lo que no escala para almacenar grandes cantidades de datos.

3.5 Matriz de latches y decodificadores

Para reducir el cableado, los latches se acomodan en una **cuadrícula**. Cada celda queda en la intersección de un cable de fila y uno de columna, y sólo se habilita cuando ambos están activos. Para activar exactamente una fila y una columna se usan **decodificadores binarios**: un decodificador de n entradas activa una sola de sus 2^n salidas, la que corresponde al número binario recibido. Con una dirección de 4 bits (2 para filas y 2 para columnas) se selecciona una de 16 celdas.

Todas las celdas comparten las líneas WRITE ENABLE, READ ENABLE y **una única línea de datos** (D_in/out). La salida de cada latch se conecta a esa línea a través de un transistor que sólo conduce cuando la celda está seleccionada y READ ENABLE = 1; el resto de las celdas quedan desconectadas (alta impedancia), por lo que no interfieren entre sí.

3.6 Memoria de bytes

Una matriz guarda un bit por dirección. Apilando 8 matrices que comparten la misma dirección, WE y RE, cada dirección guarda un byte: cada matriz aporta un bit de la palabra. Desde fuera, la memoria se ve como una lista de bytes numerados con un **bus de datos**, un **bus de direcciones**, WRITE ENABLE y READ ENABLE. El ancho del bus de direcciones determina la capacidad: con n bits se direccionan 2^n bytes (con 32 bits, 2^{32} = 4 294 967 296 bytes). Esta organización basada en latches corresponde a una memoria tipo **SRAM**.

4. Diseño en SystemVerilog

4.1 Jerarquía de módulos

El diseño se dividió en siete módulos, uno por cada nivel del video. Todos los módulos internos usan compuertas primitivas (`and`, `or`, `not`, `bufif1`) para que el código corresponda directamente al circuito.

Tabla 2. *Módulos del diseño y su correspondencia con el video.*

Nivel	Módulo	Función
1	`and_or_latch`	OR realimentada + AND con RESET invertido. Guarda 1 bit.
2	`gated_latch`	Genera SET/RESET a partir de DATA y WE.
3	`register #(WIDTH)`	WIDTH gated latches con un WE común.
4a	`binary_decoder #(N)`	Decodificador N → 2^N con salida one-hot.
4b	`memory_cell`	Gated latch + selección fila·columna + transistor de lectura (`bufif1`).
4c	`memory_matrix #(ROWS, COLS)`	Cuadrícula de celdas con una línea de datos compartida.
5	`memory_top #(DATA_WIDTH, ADDR_WIDTH)`	Módulo top: decodificadores + DATA_WIDTH matrices apiladas.

4.2 Interfaz y parámetros del módulo top

Tabla 3. *Puertos de memory_top.*

Puerto	Dirección	Ancho	Descripción
`address`	input	ADDR_WIDTH	Bus de direcciones. Bits altos → filas, bits bajos → columnas.
`write_enable`	input	1	En 1 guarda la palabra de `data` en `address`.
`read_enable`	input	1	En 1 la memoria coloca en `data` la palabra de `address`.
`data`	inout	DATA_WIDTH	Bus de datos bidireccional; en alta impedancia (Z) si `read_enable` = 0.

Tabla 4. *Parámetros de memory_top.*

Parámetro	Por defecto	Significado
`DATA_WIDTH`	8	Bits por palabra = número de matrices apiladas.
`ADDR_WIDTH`	4	Bits de dirección; capacidad = 2^ADDR_WIDTH palabras (4 → 16 bytes).
`COL_BITS` (derivado)	ADDR_WIDTH/2	Bits que van al decodificador de columnas.
`ROW_BITS` (derivado)	ADDR_WIDTH − COL_BITS	Bits que van al decodificador de filas.

4.3 Descripción de los módulos

AND-OR latch

Se modela con tres compuertas primitivas formando un lazo combinacional real, igual que el circuito del video:

```
or  u_or  (or_out, q, set);         // OR con realimentación de Q
not u_not (not_reset, reset);       // RESET invertido
and u_and (q, or_out, not_reset);   // AND que permite borrar el bit
```

Gated latch

```
not u_inv (data_n, data);
and u_set (set,   data,   write_enable);   // SET   = D  · WE
and u_rst (reset, data_n, write_enable);   // RESET = D' · WE
and_or_latch u_latch (.set(set), .reset(reset), .q(q));
```

Celda de memoria

Cada celda combina la selección por fila y columna con las señales globales de escritura y lectura. El “transistor” de salida se modela con un buffer tri-estado `bufif1`, que deja la línea en Z cuando la celda no se está leyendo:

```
and u_sel (sel,     row_sel, col_sel);       // celda seleccionada
and u_we  (cell_we, sel,     write_enable);  // escribir sólo esta celda
and u_re  (cell_re, sel,     read_enable);   // leer sólo esta celda
gated_latch u_latch (.data(data_io), .write_enable(cell_we), .q(stored_bit));
bufif1 u_tx (data_io, stored_bit, cell_re);  // transistor de lectura
```

Decodificador binario y matriz

El decodificador genera 2^N salidas con `generate`; la salida k vale 1 sólo cuando la entrada es igual a k. La matriz instancia ROWS × COLS celdas con dos ciclos `generate` anidados y conecta todas a la misma línea `data_io`.

```
for (k = 0; k < (1 << N); k++) begin : g_out
    assign out[k] = enable & (in == k);
end
```

Módulo top: memory_top

Divide la dirección entre filas y columnas, instancia los dos decodificadores (compartidos por todas las matrices) y apila DATA_WIDTH matrices, una por cada bit del bus de datos:

```
localparam int COL_BITS = ADDR_WIDTH / 2;
localparam int ROW_BITS = ADDR_WIDTH - COL_BITS;

binary_decoder #(.N(ROW_BITS)) u_row_dec (
    .in(address[ADDR_WIDTH-1 -: ROW_BITS]), .enable(1'b1), .out(row_sel));
binary_decoder #(.N(COL_BITS)) u_col_dec (
    .in(address[COL_BITS-1:0]), .enable(1'b1), .out(col_sel));

for (b = 0; b < DATA_WIDTH; b++) begin : g_plane
    memory_matrix #(.ROWS(ROWS), .COLS(COLS)) u_matrix (
        .row_sel(row_sel), .col_sel(col_sel),
        .write_enable(write_enable), .read_enable(read_enable),
        .data_io(data[b]));
end
```

4.4 Funcionamiento interno

Escritura del dato 10111100 en la dirección 1001 (ejemplo del video):

1. La dirección se divide: los bits altos `10` activan la fila 2 y los bits bajos `01` activan la columna 1.
2. En cada una de las 8 matrices, sólo la celda (2, 1) tiene activas su fila **y** su columna.

3. Al subir WRITE ENABLE, sólo esa celda recibe WE en su gated latch y copia el bit presente en su línea de datos. Las otras 15 celdas de cada matriz tienen WE = 0 y conservan su valor.

Lectura de la misma dirección:

1. Los decodificadores seleccionan la misma celda (2, 1) en las 8 matrices.
2. Con READ ENABLE = 1, sólo el `bufif1` de esa celda conduce y coloca su bit en la línea de datos.
3. Las demás celdas permanecen en alta impedancia, por lo que el bus muestra exactamente la palabra guardada.

Como los latches son sensibles a **nivel** y no a flanco, la dirección y el dato deben estar estables antes de activar WRITE ENABLE y no cambiar hasta después de desactivarlo; de lo contrario, otra celda podría capturar un valor indebido. El testbench respeta esta secuencia.

5. Verificación

Se escribieron dos testbenches autoverificables: comparan cada resultado con el valor esperado, reportan `[OK]` o `[FALLA]` y terminan con `PASS` o `FAIL`.

Tabla 5. *Pruebas realizadas.*

Testbench	Prueba	Qué verifica
`tb_latches`	AND-OR latch	SET y RESET, y que el valor se mantiene al soltarlos.
	Gated latch	Copia D con WE = 1; ignora D con WE = 0.
	Registro 8 bits	Guarda 92 = 01011100 y no cambia si la entrada cambia con WE = 0.
`tb_memory_top`	1. Ejemplo del video	Escribir 10111100 en 1001 y leerlo.
	2. Alta impedancia	Con READ ENABLE = 0 el bus queda en ZZZZZZZZ.
	3. Todas las direcciones	Escribir valores aleatorios en las 16 direcciones y leerlas contra un modelo de referencia.
	4. Aislamiento	Sobrescribir 1001 sin alterar ninguna otra dirección.
	5. LOAD R0 1001	La memoria pone el byte en el bus y un `register` lo captura, como el CPU del video.

Para comprobar la parametrización, `tb_memory_top` se ejecutó con cinco configuraciones distintas, todas con 0 errores:

Tabla 6. *Configuraciones simuladas.*

DATA_WIDTH	ADDR_WIDTH	Palabras	Matriz por bit	Resultado
8	4	16	4 × 4	PASS
1	1	2	2 × 1	PASS
4	3	8	4 × 2	PASS
16	6	64	8 × 8	PASS
32	8	256	16 × 16	PASS

6. Resultados

6.1 Simulación en Icarus Verilog

Los comandos se ejecutaron en Windows PowerShell con Icarus Verilog 12. Ambas pruebas terminaron con 0 errores.

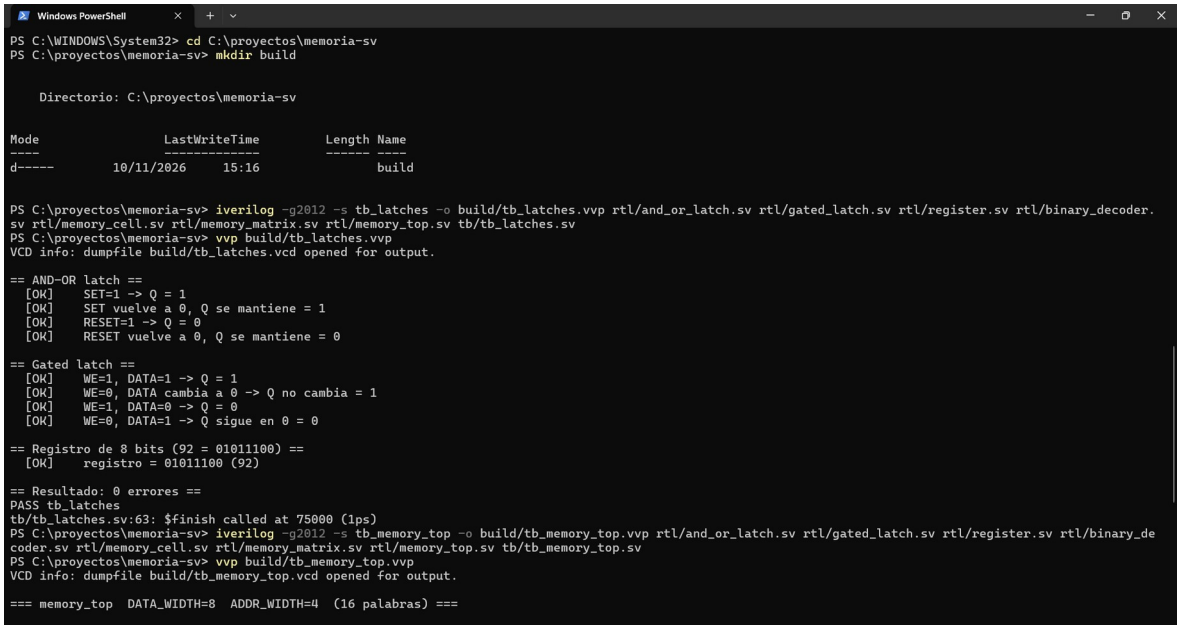


Figura 1. *Compilación y ejecución de tb_latches: AND-OR latch, gated latch y registro (PASS).*

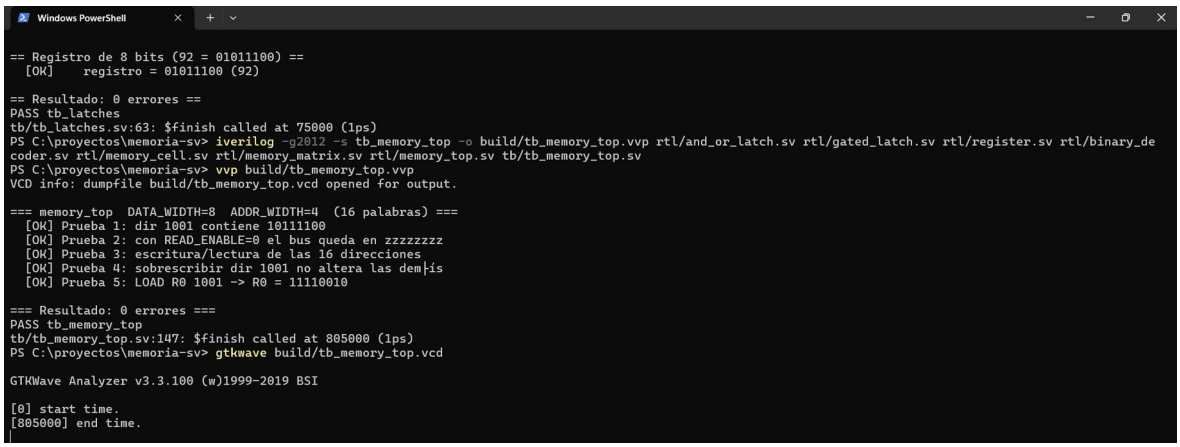


Figura 2. *Ejecución de tb_memory_top con DATA_WIDTH = 8 y ADDR_WIDTH = 4: las 5 pruebas pasan.*

En la Prueba 5 el valor de R0 varía entre corridas porque la Prueba 3 escribe datos aleatorios y la 4 sobrescribe la dirección 1001 con su complemento.

6.2 Formas de onda en GTKWave

GTKWave muestra por defecto los buses en hexadecimal: la dirección 9 equivale a `1001` y el dato `BC` equivale a `10111100`. La Figura 4 muestra los mismos buses en binario.

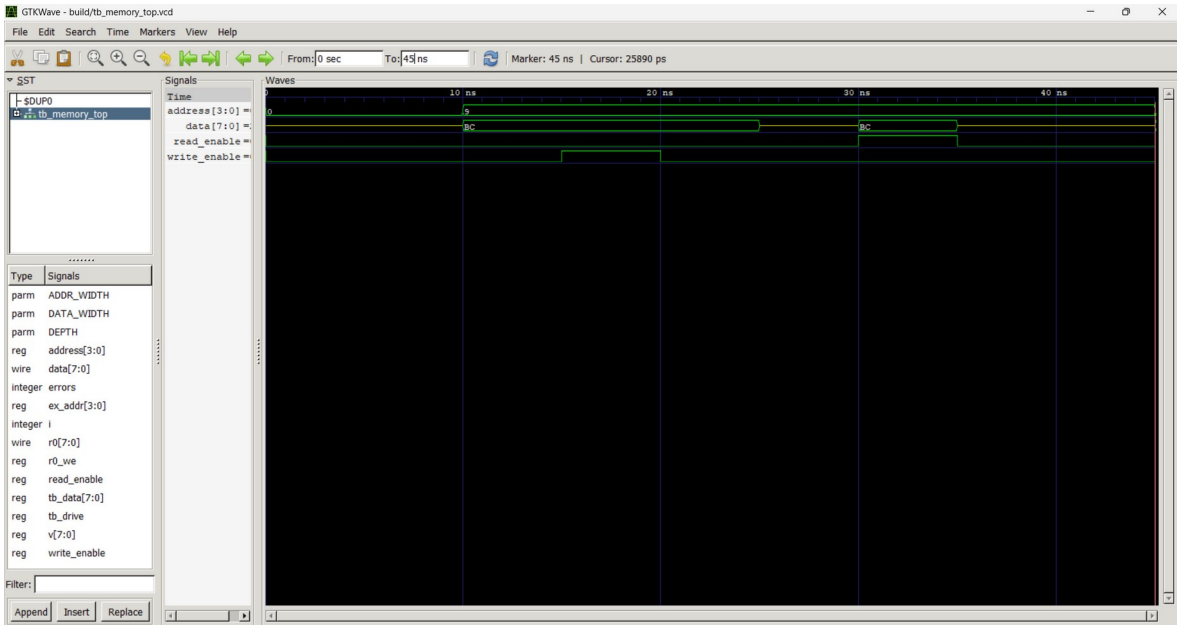


Figura 3. *Formas de onda de los primeros 45 ns en hexadecimal (address = 9, data = BC).*

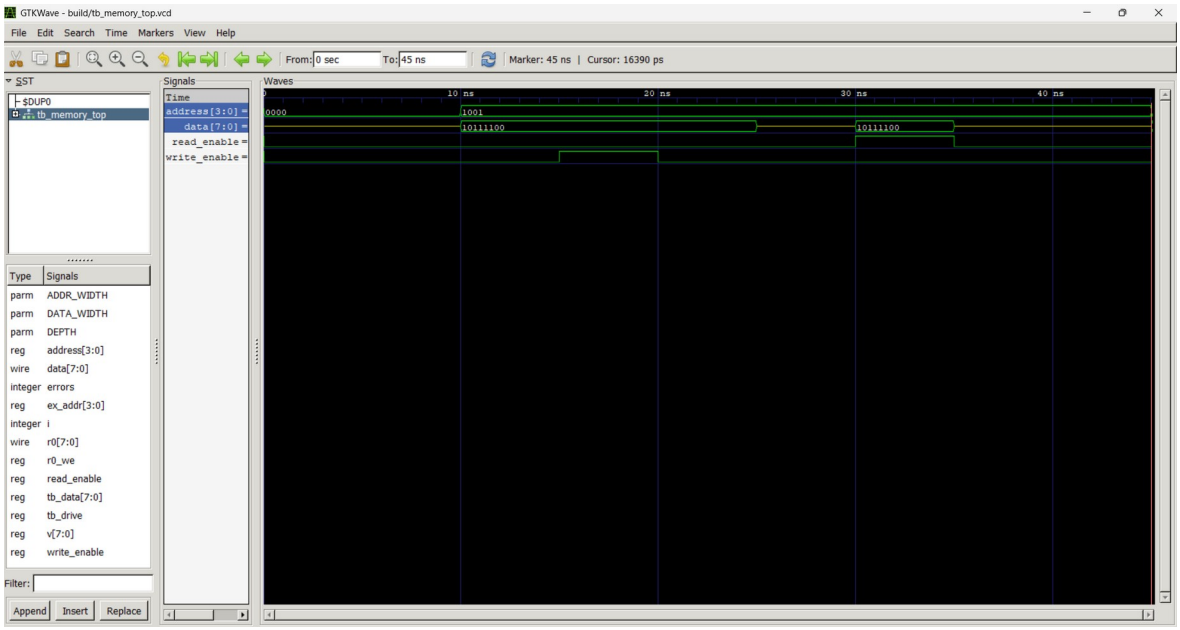


Figura 4. *Las mismas señales en binario: escritura y lectura de 10111100 en la dirección 1001.*

Tabla 7. *Interpretación de la forma de onda.*

Tiempo	Señales	Interpretación
0 – 10 ns	address = 0000, data = Z	Reposo: nadie maneja el bus.
10 ns	address = 1001, data = 10111100	El CPU/testbench coloca dirección y dato antes de escribir.
15 – 20 ns	write_enable = 1	Sólo la celda (2,1) de cada matriz captura su bit.
25 – 30 ns	data = Z (línea a la mitad)	El testbench suelta el bus; la memoria aún no lee.
30 – 35 ns	read_enable = 1, data = 10111100	La memoria, por sí sola, devuelve el byte guardado.
35 ns en adelante	data = Z	Fin de la lectura; el bus vuelve a alta impedancia.

7. Discusión

Ahorro de cableado. Para 16 bits, conectar cada latch por separado requiere 16 cables de entrada y 16 de salida (32 en total). Con la matriz basta con 4 bits de dirección, WRITE ENABLE, READ ENABLE y una línea de datos: 7 señales. La ventaja crece con el tamaño: una memoria de 2^n celdas por bit sólo necesita n líneas de dirección.

Línea de datos compartida. El uso de buffers tri-estado es lo que permite que todas las celdas compartan un solo cable sin cortocircuitos. La Prueba 2 confirma que, sin lectura activa, ninguna celda maneja el bus (estado Z).

Latch frente a flip-flop. Los latches del video son sensibles a nivel, por lo que la memoria es asíncrona y depende de que las señales respeten la secuencia correcta. En un diseño sintetizable para FPGA o ASIC se preferirían flip-flops sincronizados con reloj, o los bloques de RAM dedicados; este diseño es didáctico y reproduce fielmente el circuito.

Estado inicial. Los bits nunca escritos se leen como X en simulación, porque el latch arranca en un estado desconocido, igual que una SRAM real al encender.

SRAM frente a DRAM. Como señala el video, cada bit de esta memoria ocupa varias compuertas (varios transistores), lo que la hace rápida pero costosa en área. Por eso la memoria principal de los computadores usa DRAM, que guarda cada bit en un capacitor, y la SRAM se reserva para registros y caché.

8. Conclusiones

- Se implementó en SystemVerilog la jerarquía completa del video, desde una compuerta OR realimentada hasta una memoria de palabras, con `memory_top` como módulo top parametrizado.
- La realimentación del AND-OR latch y la lógica de habilitación del gated latch permiten guardar un bit de forma estable y escribirlo sólo cuando se desea.
- Los decodificadores de fila y columna, junto con la línea de datos compartida y los buffers tri-estado, reducen drásticamente el número de conexiones necesarias.

- Las simulaciones confirmaron escritura, lectura, aislamiento entre direcciones y alta impedancia del bus en cinco configuraciones de parámetros, todas sin errores.
- Cambiar `DATA_WIDTH` y `ADDR_WIDTH` basta para obtener memorias de distinto tamaño sin modificar la arquitectura.

9. Referencias

- Core Dumped. (s. f.). How Transistors Remember Data [Video]. YouTube. https://www.youtube.com/watch?v=rM9BjciBLmg
- Harris, D. M., & Harris, S. L. (2013). Digital Design and Computer Architecture (2.ª ed.). Morgan Kaufmann.
- IEEE. (2017). IEEE Standard for SystemVerilog (IEEE Std 1800-2017).
- Williams, S. Icarus Verilog. https://steveicarus.github.io/iverilog/

Anexo A. Estructura del repositorio y ejecución

```
memoria-sv/
├── rtl/            and_or_latch.sv, gated_latch.sv, register.sv,
│                   binary_decoder.sv, memory_cell.sv,
│                   memory_matrix.sv, memory_top.sv
├── tb/             tb_latches.sv, tb_memory_top.sv
├── docs/           waveform.png, guion_video.md
├── scripts/        plot_waves.py
├── Makefile
└── README.md
```

Comandos utilizados (Windows PowerShell, dentro de la carpeta del proyecto):

```
iverilog -g2012 -s tb_memory_top -o build/tb_memory_top.vvp rtl/and_or_latch.sv \
  rtl/gated_latch.sv rtl/register.sv rtl/binary_decoder.sv rtl/memory_cell.sv \
  rtl/memory_matrix.sv rtl/memory_top.sv tb/tb_memory_top.sv
vvp build/tb_memory_top.vvp
gtkwave build/tb_memory_top.vcd
```

Para otro tamaño de memoria se agregan `-P tb_memory_top.DATA_WIDTH=16 -P tb_memory_top.ADDR_WIDTH=6` al comando de `iverilog`.

Anexo B. Código del módulo top

```
// ============================================================================
// memory_top.sv   (MÓDULO TOP)
// ----------------------------------------------------------------------------
// Nivel 5 del video: memoria de palabras (bytes) parametrizada.
//
// Se apilan DATA_WIDTH matrices de 1 bit. Todas comparten la MISMA dirección,
// WRITE_ENABLE y READ_ENABLE; cada matriz aporta un bit de la palabra.
// Desde fuera la memoria se ve como una lista de palabras numeradas:
//
//        Data Bus  (DATA_WIDTH líneas bidireccionales)
//        Address Bus (ADDR_WIDTH líneas)  ──►  2^ADDR_WIDTH palabras
//        Write Enable, Read Enable
//
// La dirección se divide en dos mitades:
//   address[ADDR_WIDTH-1 : COL_BITS] -> decodificador de FILAS
//   address[COL_BITS-1   : 0]        -> decodificador de COLUMNAS
//
// Parámetros:
//   DATA_WIDTH : bits por palabra (8 = byte, como en el video).
//   ADDR_WIDTH : bits de dirección. Capacidad = 2^ADDR_WIDTH palabras
//                (4 -> 16 bytes como en el video; 32 -> 4 294 967 296 bytes).
//
// Operación (asíncrona, sensible a nivel, igual que los latches del video):
//   Escritura: poner address y data, luego pulsar write_enable (read_enable=0).
//   Lectura  : poner address, read_enable = 1; la memoria maneja data.
//   Reposo   : read_enable = 0 -> data queda en alta impedancia ('z).
// ============================================================================
`timescale 1ns/1ps

module memory_top #(
    parameter int DATA_WIDTH = 8,
    parameter int ADDR_WIDTH = 4
) (
    input  logic [ADDR_WIDTH-1:0] address,
    input  logic                  write_enable,
    input  logic                  read_enable,
    inout  wire  [DATA_WIDTH-1:0] data          // bus de datos bidireccional
);
    // ------------------------------------------------------------------
    // División de la dirección en fila / columna
```

```
    // ------------------------------------------------------------------
    localparam int COL_BITS = ADDR_WIDTH / 2;
    localparam int ROW_BITS = ADDR_WIDTH - COL_BITS;
    localparam int ROWS     = 1 << ROW_BITS;
    localparam int COLS     = 1 << COL_BITS;
    localparam int DEPTH    = 1 << ADDR_WIDTH;   // número de palabras

    logic [ROWS-1:0] row_sel;
    logic [COLS-1:0] col_sel;

    // ------------------------------------------------------------------
    // Decodificadores binarios (compartidos por todas las matrices,
    // porque todas reciben la misma dirección)
    // ------------------------------------------------------------------
    binary_decoder #(.N(ROW_BITS)) u_row_dec (
        .in     (address[ADDR_WIDTH-1 -: ROW_BITS]),
        .enable (1'b1),
        .out    (row_sel)
    );

    generate
        if (COL_BITS > 0) begin : g_coldec
            binary_decoder #(.N(COL_BITS)) u_col_dec (
                .in     (address[COL_BITS-1:0]),
                .enable (1'b1),
                .out    (col_sel)
            );
        end else begin : g_nocol
            assign col_sel = 1'b1;   // ADDR_WIDTH = 1: una sola columna
        end
    endgenerate

    // ------------------------------------------------------------------
    // DATA_WIDTH matrices de 1 bit apiladas (una por bit del bus)
    // ------------------------------------------------------------------
    genvar b;
    generate
        for (b = 0; b < DATA_WIDTH; b++) begin : g_plane
            memory_matrix #(
                .ROWS (ROWS),
                .COLS (COLS)
            ) u_matrix (
                .row_sel      (row_sel),
                .col_sel      (col_sel),
                .write_enable (write_enable),
                .read_enable  (read_enable),
                .data_io      (data[b])
            );
        end
    endgenerate

    // ------------------------------------------------------------------
    // Verificación de parámetros
    // ------------------------------------------------------------------
    initial begin
        if (DATA_WIDTH < 1) $fatal(1, "DATA_WIDTH debe ser >= 1");
        if (ADDR_WIDTH < 1) $fatal(1, "ADDR_WIDTH debe ser >= 1");
    end

endmodule
```